



受控异常与自动释放的资源

北京理工大学计算机学院
金旭亮

受控异常



出怪事了!

- 请看以下代码，它们完全符合Java语法规则，但事实是它们不能通过编译：

```
public class TestThrows {  
    public static void main(String[] args) {  
        FileInputStream fis = new FileInputStream("a.txt");  
    }  
}
```

为什么?



原因

查看一下JDK中FileInputStream的源码，会发现它的构造方法后面跟着一个“奇怪”的声明：

```
public FileInputStream( @NotNull String name) throws FileNotFoundException {  
    this(name != null ? new File(name) : null);  
}
```



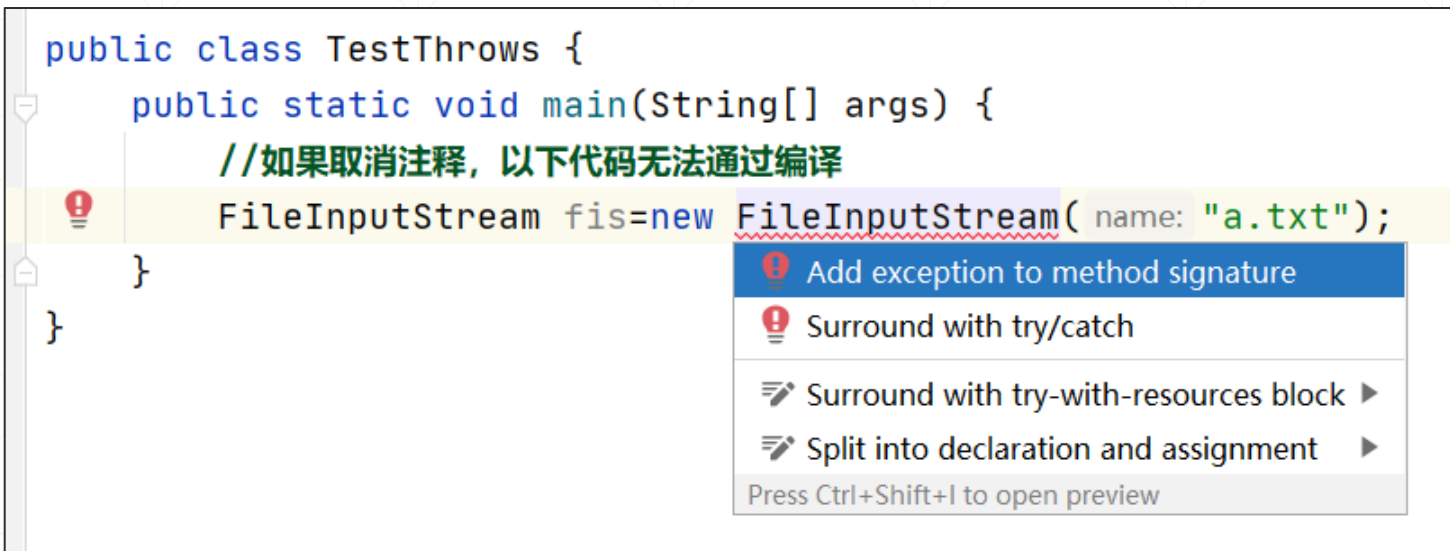
受控的异常



throws语句表明某方法中可能出现某种（或多种）异常，但它自己不能处理这些异常，而需要由调用者来处理。

修正代码以通过编译的两种方式

```
public class TestThrows {  
    public static void main(String[] args) {  
        //如果取消注释，以下代码无法通过编译  
        FileInputStream fis=new FileInputStream( name: "a.txt");  
    }  
}
```



当一个方法声明中包含有throws子句时，需要在调用此方法的代码中使用try/catch/finally进行捕获，或者是重新对其进行声明，否则编译时报错。

修改示例代码以通过编译

方式一:

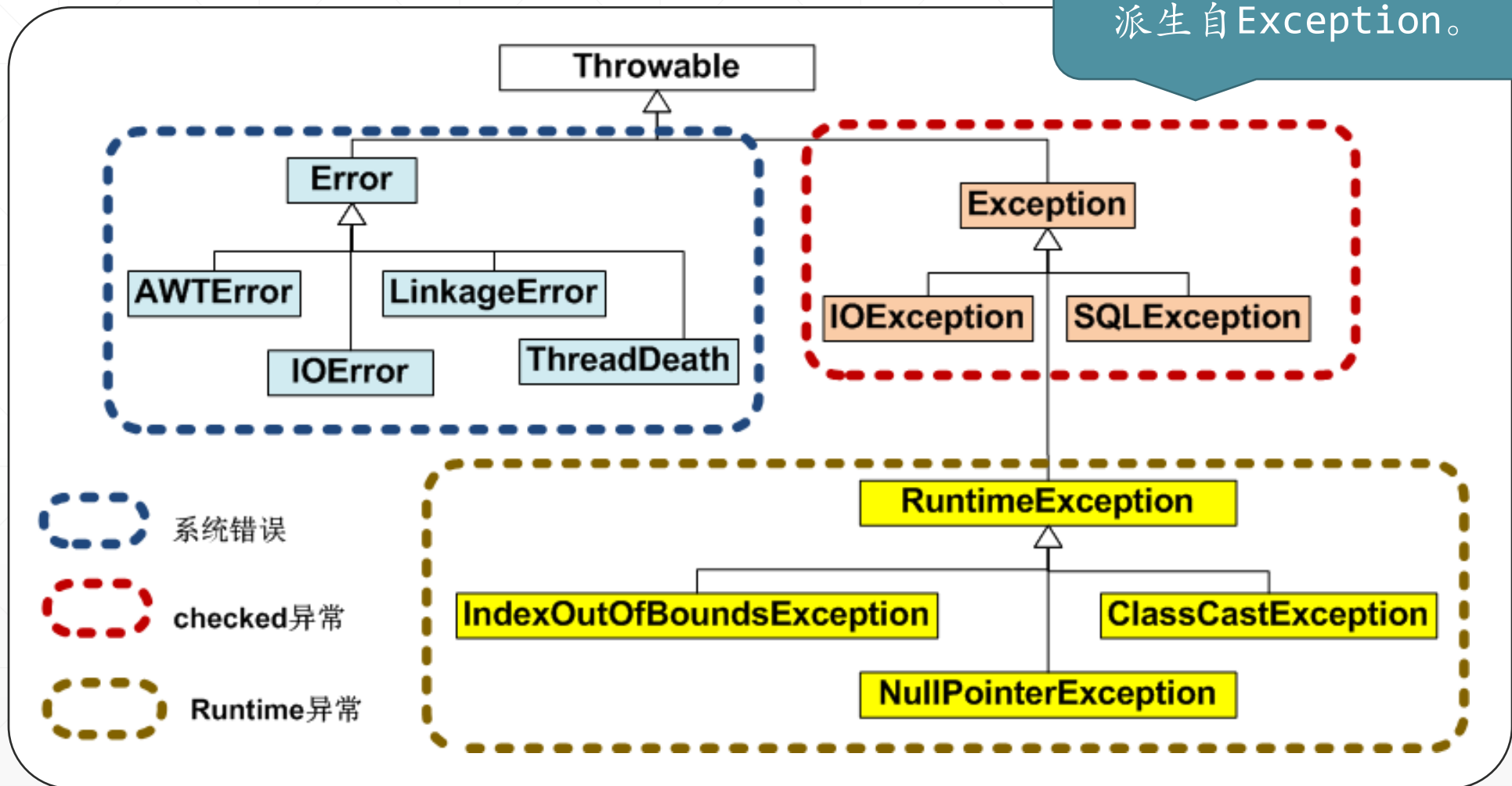
```
public class TestThrows {  
    public static void main(String[] args)  
        throws FileNotFoundException  
    {  
        FileInputStream fis = new FileInputStream("a.txt");  
    }  
}
```

方式二:

```
public class TestThrows {  
    public static void main(String[] args) {  
        try {  
            FileInputStream fis = new FileInputStream("a.txt");  
        } catch (FileNotFoundException e) {  
            e.printStackTrace();  
        }  
    }  
}
```

复习：JDK中与异常相关的类

受控的异常，通常直接派生自Exception。



Java中的异常处理策略

受控异常
(直接派生自Exception)

在编译阶段进行检查，不符合要求的代码，拒绝编译

系统错误
(派生自Error)

运行时异常
(派生自RuntimeException)

编译器在编译阶段不负责检查此类异常，而由程序员编写代码在程序运行时进行捕获

阶段小结：受控与不受控的异常

- throws语句中声明的异常称为**受控 (checked) 的异常**，通常直接派生自Exception类。
- RuntimeException（其基类为Exception）和Error（基类为Throwable）称为**非受控的异常**。这种异常不用（当然也可以）在throws语句中声明。
- 当你调用一个使用了throws语句声明要抛出异常的方法时，受控的异常必须被捕获，否则将无法通过编译。而非受控异常可以不被捕获，代码会顺利编译，但有可能在运行期间抛出这个异常。
- 所以从健壮性角度考虑，调用有抛出异常声明的方法时，应该妥善处理其抛出的异常。
- CheckedExceptionDemo.java示例展示了上述两种异常的特性。

受控与不受控的异常示例

```
public class CheckedExceptionDemo {  
    public static void main(String[] args) {  
        try {  
            var buf = new BufferedReader(  
                new InputStreamReader(System.in));  
            System.out.print("请输入整数: ");  
            //readLine()方法声明会抛出IOException  
            //parseInt()方法声明会抛出NumberFormatException  
            int input = Integer.parseInt(buf.readLine());  
            System.out.println("input x 10 = " + (input*10));  
        }  
        //以下异常处理语句块是必须的，否则无法通过编译  
        //因为IOException的基类是Exception  
        catch(IOException e) {  
            System.out.println("I/O错误");  
        }  
        //以下异常处理语句块可以省略，不影响编译，  
        //因为NumberFormatException的基类是RuntimeException  
        //但可能会在运行时出错，因为用户可能会输入非法的数字  
        catch(NumberFormatException e) {  
            System.out.println("输入必须为整数");  
        }  
    }  
}
```

请仔细地阅读示例源码及相关的注释，以便比较全面地掌握相关的知识。

CheckedExceptionDemo.java

编写自己的“受控”方法

当编写自己的代码时，如果你的代码需要引发“受控异常”，则应该使用throws关键字进行声明：

```
void myMethod() throws MyCheckedException
{
    //...
}
```

这样干了之后，外界调用这个方法时，要不用try/catch捕获它，要不就是再次抛出去。

抛出多个受控异常的方法

- 一个方法可以声明抛出多个异常

```
int func(float h) throws OneException, TwoException {  
    //...  
}
```

- 注意一个Java异常处理中的一个比较独特的地方：

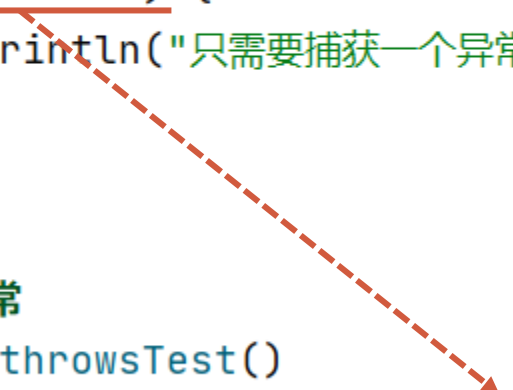


当一个方法声明抛出多个异常时，在此方法调用语句处只要catch其中任何一个异常，代码就可以顺利编译。

调用抛出多个异常的方法

ThrowMultiExceptionsDemo.java示例展示了相关特性。

```
public class ThrowMultiExceptionsDemo {  
    public static void main(String[] args) {  
        try {  
            throwsTest();  
        } catch (IOException e) {  
            System.out.println("只需要捕获一个异常就可以了");  
        }  
    }  
  
    //此方法声明会抛出两个异常  
    private static void throwsTest()  
        throws ArithmeticException, IOException {...}  
}
```

A red dashed arrow originates from the `IOException` parameter in the `catch` block and points towards the `IOException` in the `throws` clause of the `throwsTest` method, illustrating that the catch block handles one of the exceptions declared in the method signature.

子类抛出受控异常的限制

一个子类的throws子句抛出的异常，不能是其基类同名方法抛出的异常对象的父类。

```
public class OverrideThrows {  
    public void test() throws IOException {  
        var fis = new FileInputStream(name: "a.txt");  
    }  
}  
  
class Sub extends OverrideThrows {  
    //你可以试一试,  
    //如果test方法声明抛出了比父类方法更大的异常,比如Exception  
    //则代码将无法编译.....  
    public void test() throws FileNotFoundException {  
        //...  
    }  
}
```

OverrideThrows.java

Java 7 中的新特性

- Java 7 及以后的版本，允许在一个catch块中捕获多个异常。
- 示例代码如下：

```
try {  
    //...  
    throw new SocketException();  
}  
catch (SocketException | SecurityException | NullPointerException e) {  
    //exception handler  
}
```



使用这种方式捕获异常，要求异常捕获列表中的异常类型之间应该没有父子关系，比如，无法将FileNotFoundException与IOException放在异常捕获表达式中。

在实际开发合理使用异常抛出声明

```
public static void throwChecked(int a) throws Exception {  
    if (a > 0) {  
        // 自行抛出Exception异常  
        // 该代码必须处于try块里, 或处于带throws声明的方法中  
        throw new Exception("a的值大于0, 不符合要求");  
    }  
}  
  
public static void throwRuntime(int a) {  
    if (a > 0) {  
        // 自行抛出RuntimeException异常, 既可以显式捕获该异常  
        // 也可完全不理睬该异常, 把该异常交给该方法调用者处理  
        throw new RuntimeException("a的值大于0, 不符合要求");  
    }  
}
```

ThrowRuntimeExceptionTest.java

```
public static void main(String[] args) {  
    try {  
        // 调用声明抛出Checked异常的方法, 要么显式捕获该异常  
        // 要么在main方法中再次声明抛出  
        throwChecked(a: -3);  
    } catch (Exception e) {  
        System.out.println(e.getMessage());  
    }  
    // 调用声明抛出Runtime异常的方法既可以显式捕获该异常,  
    // 也可不理睬该异常  
    throwRuntime(a: 3);  
}
```


关于Java“受控异常”特性引发的争议

在实际开发中，大部分的方法总是很难明确地知道如何处理异常，这给程序员带来了很重的心智负担，因为你不知道外界可能会在哪种场景下，怎样使用这些方法代码，另外，当方法嵌套多层调用时，可能会抛出N个异常，处理这些异常很讨厌。



因此很多程序员在开发被别人调用的代码，或者是调用其他人的代码时，通常都是简单地在方法声明处直接再次抛出异常，或者到处放置try/catch语句，导致整个代码显得臃肿不堪。



在大部分时候推荐使用 Runtime异常，而不使用 Checked异常。



自动释放的资源

概述



通常情况下，当我们创建一个普通的Java对象时，需要占用的主要是内存资源，JVM会帮助我们管理好这些资源，程序员无须费心。



但有些特殊的对象，它不光要占用内存资源，还需占用一些由操作系统直接管理的特殊资源，这些特殊的资源，JVM是不能自动管理与释放的，只能由程序员写代码来负责释放。



这些特殊的对象，最典型的有“文件流对象”，“数据库连接对象”，“Socket对象”等等，当你不再使用它们时，应该显式地调用它们的close()方法释放它们。否则，就可能会引发资源泄露。

什么叫“资源泄露”？



计算机所拥有的资源数量是有限的，因此，如果某个程序占用了过多的资源，不仅不释放，还在不断地申请新的资源，那么迟早这台计算机就会因为资源耗尽而无法继续工作。



假设某应用程序向操作系统申请并占用了特定的资源，用完之后，本应及时归还操作系统，但这个应用程序并没有这么做。对有限的资源“自己不用也不让其他人用”，这种情况称为“**资源泄露**”。



实际开发中的“资源泄露”，多数是由程序代码中的缺陷所引发的，避免发生这种情况，是程序员的职责。

如何避免出现“资源泄漏”？



应用程序在申请并使用完资源之后，应该立即将这些资源归还操作系统。



在Java中，这些资源所对应的对象，通常都提供有一个`close()`方法，程序员应该在用完这些对象之后，立即调用它的`close()`方法释放这些资源。

JDK中用于读取文件内容的`FileReader`对象就是一个需要及时关闭的资源，下面我们来看一下，Java中是怎样释放它的.....

传统Java读取文件代码

为了能及时地释放资源，我们需要在finally块中调用FileReader对象的close()方法。

由于FileReader的构造函数声明会抛出FileNotFoundException，它的close()方法声明会抛出IOException，所以，为了保证打开的资源一定可以释放，我们不得不编写两重嵌套的try/catch结构。

这样的代码非常地别扭！

```
static void traditionalReadFile() {  
    File file = new File( pathname: "hello.txt");  
    FileReader reader = null;  
    try {  
        reader = new FileReader(file);  
        //使用reader从文件中读取数据  
    } catch (FileNotFoundException e) {  
        e.printStackTrace();  
    } finally {  
        if (reader != null) {  
            try {  
                reader.close();  
            } catch (IOException e) {  
                e.printStackTrace();  
            }  
        }  
    }  
}
```

ReadFileDemo.java

简化代码

```
static void readFileUseTryWithResources() {  
    File file = new File( pathname: "hello.txt");  
    try (var reader = new FileReader(file)) {  
        //使用reader从文件中读取数据  
    } catch (FileNotFoundException e) {  
        e.printStackTrace();  
    } catch (IOException e) {  
        e.printStackTrace();  
    }  
}
```

ReadFileDemo.java

使用Java 7所引入的try with resources特性，可将前述两层嵌套的try/catch结构简化为一层，并且不再需要显式地调用close()方法，这个close()方法会在程序执行流程离开try语句块时被自动地调用。

解析Java 7特性：自动释放的资源-1

- try-with-resources：当程序执行流程离开try语句块时，会自动地调用被监控资源对象的close()方法：

```
try ( ClassName theObject = new ClassName() ) {  
    // 使用theObject干活.....  
}  
catch ( Exception e ) {  
    //.....  
}
```



要求ClassName指定的类型必须实现AutoCloseable接口（或Closeable接口）。

解析Java 7特性：自动释放的资源-2

- AutoCloseable接口的定义如下：

```
public interface AutoCloseable {  
    void close() throws Exception;  
}
```



当程序执行流程离开 try-with-resources 结构时，AutoCloseable接口所定义的close()方法会被自动调用，因此，无需在finally子句中显式调用close()方法。

使用JDK中的文件组件读取与写入文件



```
String fileName = "hello.txt";
var file = new File( pathname: "hello.txt");
if (!file.exists()) {
    try (var outputStream = new FileOutputStream(file)) {
        outputStream.write("Hello,World!"
            .getBytes(Charset.defaultCharset()));
        System.out.println("数据文件已创建");
    } catch (IOException e) {
        e.printStackTrace();
    }
}
```

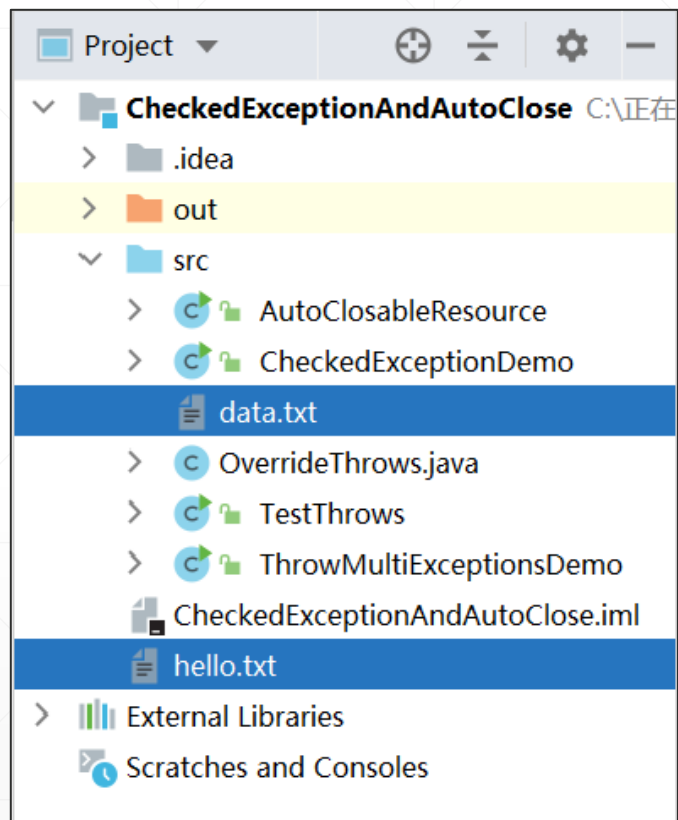
允许“监控”多个资源
对象.....

```
try (FileReader reader = new FileReader(file);  
    BufferedReader br = new BufferedReader(reader)) {  
    var text :String = br.readLine();  
    System.out.println(text);  
} catch (IOException e) {  
    e.printStackTrace();  
}
```

AutoClosableResource.java

JDK为文件的读取提供了非常丰富的组件，相关的内容将在“文件与流”这部分进行介绍。

“顺便”多学一点知识



在IntelliJ中，最顶层的目录，称为“项目文件夹”，放在其中的文件，可以直接通过文件名访问：

```
var file = new File("hello.txt");
```

子文件夹中的文件，则需要指定路径名。比如放在src下的data.txt文件，则可以通过“src/data.txt”访问到它。



实际开发中，还可以在文件路径中使用两个特殊的符号，比如“.”表示当前文件夹，“..”表示上层文件夹。

下一讲



介绍“自定义异常与异常处理链”